

ENRIQUE CRESPO RAMIREZ

Arquitectura en la Nube — Práctica: WordPress + ngrok
(WSL/Ubuntu)

Visión general (qué vamos a montar y por qué)

- **Apache (puerto 80):** servidor web que entregará WordPress. Lo elegimos porque la práctica lo pide y es estándar.
- **MySQL:** base de datos relacional donde WordPress guarda todo (usuarios, posts, ajustes...).
- **PHP + extensiones:** motor que ejecuta WordPress (que está escrito en PHP).
- **WordPress:** CMS que correrá sobre Apache+PHP conectado a MySQL.

- **ngrok:** túnel https seguro que expone tu `localhost:80` a Internet sin abrir puertos en tu router.

🌱 Infraestructura del entorno			
Servicio	Función	Puerto	Motivo
Apache2	Servidor web para WordPress	80	Requerido por la práctica y necesario para ngrok
MySQL Server	Base de datos de WordPress	3306	Puerto estándar MySQL
PHP + módulos	Ejecución de WordPress	—	Motor de ejecución
ngrok	Túnel HTTPS público	—	Acceso externo seguro
WordPress	CMS que integra Apache + MySQL	—	Proyecto final
↓			

PARTE 1 — Instalar el stack LAMP

1.1 Actualizar sistema

sudo apt update

sudo apt upgrade -y

Por qué: evitas fallos por dependencias antiguas y te aseguras parches de seguridad antes de instalar servicios base.

```
kike@Double-KK:/mnt/c/Windows/System32$ sudo apt update
[sudo] password for kike:
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Get:2 https://dl.cloudsmith.io/public/caddy/stable/deb/ubuntu any-version InRelease [14.8 kB]
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 141 kB in 2s (72.2 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
15 packages can be upgraded. Run 'apt list --upgradable' to see them.
kike@Double-KK:/mnt/c/Windows/System32$ sudo apt update -y
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Get:4 https://dl.cloudsmith.io/public/caddy/stable/deb/ubuntu any-version InRelease [14.8 kB]
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Fetched 14.8 kB in 8s (1752 B/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
15 packages can be upgraded. Run 'apt list --upgradable' to see them.
kike@Double-KK:/mnt/c/Windows/System32$
```

1.3 Instalar Apache y configurar el puerto 80

sudo apt install apache2 -y

```
kike@Double-KK:/mnt/c/Windows/System32$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-10-19 10:29:53 CEST; 58min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 163 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Process: 423 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
  Main PID: 337 (apache2)
    Tasks: 7 (limit: 37958)
   Memory: 17.6M (peak: 32.7M)
      CPU: 288ms
   CGroup: /system.slice/apache2.service
           └─ 337 /usr/sbin/apache2 -k start
              468 /usr/sbin/apache2 -k start
              470 /usr/sbin/apache2 -k start
              471 /usr/sbin/apache2 -k start
              472 /usr/sbin/apache2 -k start
              473 /usr/sbin/apache2 -k start
             1002 /usr/sbin/apache2 -k start

Oct 19 10:29:53 Double-KK systemd[1]: Starting apache2.service - The Apache HTTP Server...
Oct 19 10:29:53 Double-KK systemd[1]: Started apache2.service - The Apache HTTP Server.
Oct 19 10:29:54 Double-KK systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Oct 19 10:29:54 Double-KK systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
kike@Double-KK:/mnt/c/Windows/System32$
```

Por qué: Apache será el servidor que entregue WordPress por HTTP.

Configurar Apache para usar el puerto 80

sudo nano /etc/apache2/ports.conf

```
kike@Double-KK: /mnt/c/Windows/System32
GNU nano 7.2
# If you just change the port or add more ports here, you
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 80
<IfModule ssl_module>
#     Listen 443
</IfModule>

<IfModule mod_gnutls.c>
#     Listen 443
</IfModule>
```

Después edita el sitio por defecto, y que contenga el puerto 80:

sudo nano /etc/apache2/sites-enabled/000-default.conf

```
kike@Double-KK: /mnt/c/Windows/System32
GNU nano 7.2
<VirtualHost *:80>
# The ServerName directive sets the request scheme, hostname and port that
# the server uses to identify itself. This is used when creating
# redirection URLs. In the context of virtual hosts, the ServerName
# specifies what hostname must appear in the request's Host: header to
# match this virtual host. For the default virtual host (this file) this
# value is not decisive as it is used as a last resort host regardless.
# However, you must set it for any further virtual host explicitly.
#ServerName www.example.com

ServerAdmin webmaster@localhost
DocumentRoot /var/www/html

# Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
# error, crit, alert, emerg.
# It is also possible to configure the loglevel for particular
# modules, e.g.
#LogLevel info ssl:warn

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined

# For most configuration files from conf-available/, which are
# enabled or disabled at a global level, it is possible to
# include a line for only one particular virtual host. For example the
# following line enables the CGI configuration for this host only
# after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

sudo systemctl restart apache2

Comprueba que ahora escucha correctamente el 80:

```
kike@Double-KK:/var/www/html$ sudo lsof -i :80
COMMAND  PID    USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
apache2  3448   root   4u  IPv6  29871      0t0  TCP *:http (LISTEN)
apache2  3450 www-data 4u  IPv6  29871      0t0  TCP *:http (LISTEN)
apache2  3451 www-data 4u  IPv6  29871      0t0  TCP *:http (LISTEN)
apache2  3452 www-data 4u  IPv6  29871      0t0  TCP *:http (LISTEN)
apache2  3453 www-data 4u  IPv6  29871      0t0  TCP *:http (LISTEN)
apache2  3454 www-data 4u  IPv6  29871      0t0  TCP *:http (LISTEN)
apache2  3466 www-data 4u  IPv6  29871      0t0  TCP *:http (LISTEN)
kike@Double-KK:/var/www/html$
```

2) MySQL listo y en el puerto 3306 Instalar MySQL (base de datos)

`sudo apt install mysql-server -y`

`sudo mysql_secure_installation`

Qué hace:

- Instala el servidor MySQL.
- `mysql_secure_installation` te guía para endurecer: contraseña de root, borrar usuarios anónimos, deshabilitar accesos remotos inseguros, etc.

Respuestas recomendadas:

Pregunta	Respuesta
¿Validación de contraseñas?	N
¿Cambiar contraseña de root?	Y
Contraseña (ejemplo)	Admin123!
Restantes preguntas	Y

2.1 Crear la base de datos y el usuario de WordPress

sudo systemctl status mysql --no-pager

sudo ss -ltnp | grep 3306 || sudo lsof -i :3306

sudo mysql <<'SQL'

DROP DATABASE IF EXISTS test;

DELETE FROM mysql.db WHERE Db='test' OR Db='test_%';

FLUSH PRIVILEGES;

SQL

```
kike@Double-KK:/var/www/html$ sudo systemctl status mysql --no-pager
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-10-19 11:40:12 CEST; 2min 21s ago
   Process: 4229 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
   Main PID: 4238 (mysqld)
     Status: "Server is operational"
    Tasks: 39 (limit: 37958)
   Memory: 353.1M (peak: 378.3M)
      CPU: 914ms
   CGroup: /system.slice/mysql.service
           └─4238 /usr/sbin/mysqld

Oct 19 11:40:11 Double-KK systemd[1]: Starting mysql.service - MySQL Community Server...
Oct 19 11:40:12 Double-KK systemd[1]: Started mysql.service - MySQL Community Server.
kike@Double-KK:/var/www/html$ sudo ss -ltnp | grep 3306 || sudo lsof -i :3306
LISTEN 0      151          127.0.0.1:3306      0.0.0.0:*        users:(("mysqld",pid=4238,fd=23))

LISTEN 0      70          127.0.0.1:33060     0.0.0.0:*        users:(("mysqld",pid=4238,fd=21))

kike@Double-KK:/var/www/html$ sudo mysql <<'SQL'
CREATE DATABASE IF NOT EXISTS wordpress;
CREATE USER IF NOT EXISTS 'wpuser'@'localhost' IDENTIFIED BY 'WordPress123!';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'localhost';
FLUSH PRIVILEGES;
SQL
kike@Double-KK:/var/www/html$ mysql -u wpuser -p'WordPress123!' -e "SHOW DATABASES LIKE 'wordpress';"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Database (wordpress) |
+-----+
| wordpress             |
+-----+

kike@Double-KK:/var/www/html$ sudo mysql -e "SHOW GRANTS FOR 'wpuser'@'localhost';"
+-----+
| Grants for wpuser@localhost |
+-----+
| GRANT USAGE ON *.* TO `wpuser`@`localhost` |
| GRANT ALL PRIVILEGES ON `wordpress`.* TO `wpuser`@`localhost` |
+-----+

kike@Double-KK:/var/www/html$
```

3) Descargar e instalar WordPress (core limpio)

```
cd /tmp
```

```
wget https://wordpress.org/latest.tar.gz
```

```
tar -xzf latest.tar.gz
```

```
sudo rm -rf /var/www/html/* ( Limpia la raiz publica)
```

```
sudo cp -r wordpress/* /var/www/html/ Copia WordPress
```

Qué hace: deja el código WP en la raíz servida por Apache.

Validación: `ls /var/www/html` debe

mostrar `wp-admin`, `wp-content`, `wp-includes`, etc.

4) Permisos correctos para Apache

```
sudo chown -R www-data:www-data /var/www/html/
```

```
sudo chmod -R 755 /var/www/html/
```

Por qué: `www-data` (usuario del servicio web) debe poder leer/ejecutar el código y escribir en algunas rutas en el futuro (p. ej., subidas). 755 es seguro para arranque inicial.

PUNTO 5 — Configurar wp-config.php (la pieza que une todo)

Hasta ahora, hemos montado tres cosas:

- 1- Apache (servidor web)
- 2- MySQL (base de datos)
- 3- PHP (el lenguaje que ejecuta WordPress)

Ahora falta **enlazarlas**: que WordPress sepa **cómo conectarse** a la base de datos donde guardará todo su contenido (usuarios, entradas, configuraciones...).

Esa información se guarda en el archivo **wp-config.php**.

¿Qué es wp-config.php?

Es un archivo clave de configuración dentro del directorio `/var/www/html/` que indica a WordPress:

- Dónde está la base de datos (`DB_HOST`)
- Qué base de datos debe usar (`DB_NAME`)
- Con qué usuario se conecta (`DB_USER`)
- Qué contraseña usa ese usuario (`DB_PASSWORD`)
- Y, más adelante, qué URL pública tendrá (`WP_HOME` y `WP_SITEURL`)

Sin este archivo, WordPress no puede funcionar porque no sabría dónde guardar ni leer sus datos.

Paso 5.1 — Crear wp-config.php

Primero, copiamos el archivo de ejemplo que WordPress trae por defecto:

```
sudo cp /var/www/html/wp-config-sample.php /var/www/html/wp-config.php
```

Qué hace:

- Crea un nuevo archivo llamado `wp-config.php` basado en la plantilla de ejemplo.
- Este nuevo archivo será el que WordPress lea al iniciar.

Paso 5.2 — Editar el archivo con tus credenciales

```
sudo nano /var/www/html/wp-config.php
```



```

*/
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress' );

/** Database username */
define( 'DB_USER', 'kike' );

/** Database password */
define( 'DB_PASSWORD', 'kike123_' );

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );

/**#@+
 * Authentication unique keys and salts.
 *
 * Change these to different unique phrases! You can generate these using
 * the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}.
 *
 * You can change these at any point in time to invalidate all existing cookies.
 * This will force all users to have to log in again.
 *
 * @since 2.6.0

```

DB_NAME

Le dice a WordPress qué base de datos usar.

DB_USER

Usuario con permisos sobre la BD.

DB_PASSWORD

Contraseña del usuario.

DB_HOST

Dónde está la base de datos.

PUNTO 6 — Instalación inicial de WordPress desde el navegador

Hasta ahora, tenemos:

- Apache activo sirviendo /var/www/html
- MySQL con la base de datos wordpress
- wp-config.php con tus credenciales (DB_NAME=wordpress, DB_USER=kike, DB_PASSWORD=kike123)
- PHP funcionando correctamente

Así que WordPress ya tiene **todo lo necesario para arrancar**

Qué hace WordPress ahora

Cuando visitas tu sitio por primera vez, WordPress:

1. Lee el archivo wp-config.php
2. Se conecta a MySQL con las credenciales que le diste
3. Crea las tablas necesarias dentro de la base wordpress (usuarios, posts, etc.)
4. Te pide configurar un **usuario administrador** y los datos iniciales del sitio.

Paso 6.1 — Reinicia Apache (por seguridad)

La pagina de localhost no funcionaba correctamente!!

EXPLICACIÓN TÉCNICA — Qué ocurrió y por qué fallaba

1. **Apache sí estaba funcionando correctamente**, pero al intentar acceder a WordPress aparecía el mensaje “*Requirements Not Met*” o no se ejecutaban los archivos .php.
Esto sucedía porque **PHP no tenía instaladas las extensiones necesarias** para conectarse con MySQL ni para procesar correctamente el código PHP de WordPress.
2. Cuando intentaste instalar los paquetes genéricos (php, php-mysql, php-curl, etc.), **Ubuntu no los reconocía**:

E: Unable to locate package php-curl

Esto ocurre en **Ubuntu 24.04 / WSL** porque el sistema utiliza PHP **8.3** por defecto, y las extensiones ya no se llaman php-xxx, sino php8.3-xxx.

Por ejemplo:

- a. php-mysql → ahora es php8.3-mysql
 - b. php-curl → ahora es php8.3-curl
 - c. libapache2-mod-php → ahora es libapache2-mod-php8.3
3. Debido a eso, Apache estaba sirviendo el HTML (por eso veías tu página de prueba), pero **no podía interpretar el código PHP**, y WordPress no podía conectarse con MySQL.

Identificamos la versión activa de PHP

`php -v`

Resultado: PHP 8.3.6

Esto nos confirmó que debíamos instalar los módulos específicos de la versión 8.3.

Instalamos todas las extensiones PHP 8.3 necesarias para WordPress

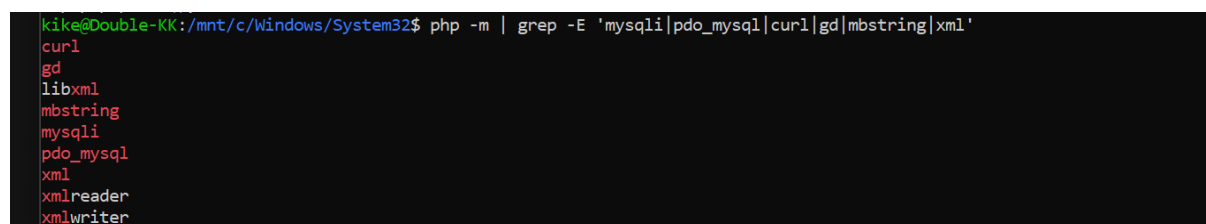
```
sudo apt install -y php8.3 php8.3-mysql libapache2-mod-php8.3 \
    php8.3-curl php8.3-gd php8.3-mbstring php8.3-xml php8.3-intl
php8.3-zip
```

Activamos PHP 8.3 en Apache

```
sudo a2enmod php8.3
sudo systemctl restart apache2
```

Verificamos que las extensiones se activaron

```
php -m | grep -E 'mysqli|pdo_mysql|curl|gd|mbstring|xml'
```



```
kike@Double-KK:/mnt/c/Windows/System32$ php -m | grep -E 'mysqli|pdo_mysql|curl|gd|mbstring|xml'
curl
gd
libxml
mbstring
mysqli
pdo_mysql
xml
xmlreader
xmlwriter
```

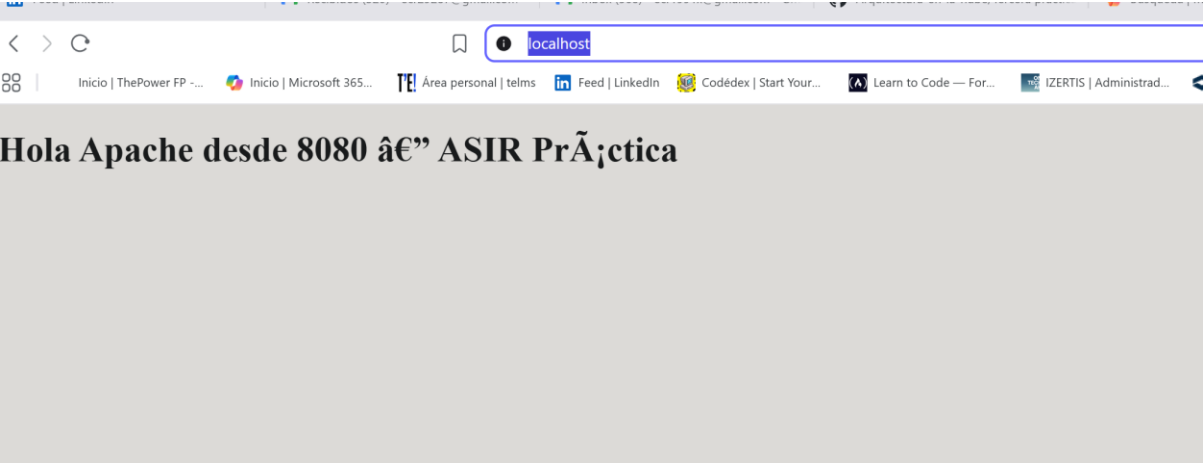
Devolvió los módulos activos, confirmando que PHP ya tiene todo lo que WordPress necesita.

Creamos un archivo `phpinfo.php` para comprobar que Apache ejecuta PHP

`echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/phpinfo.php`

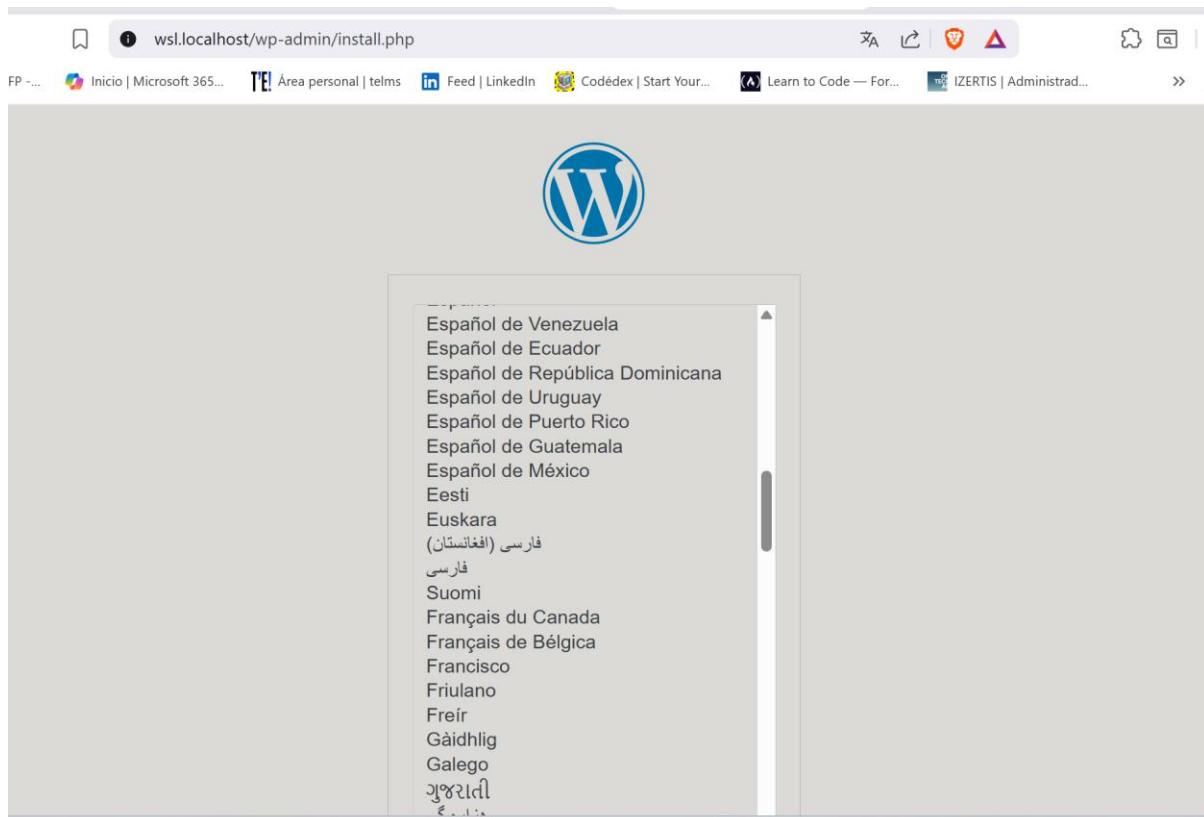
PHP Version 8.3.6	
	
System	Linux Double-KK 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64
Build Date	Jul 14 2025 18:30:55
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.3/apache2
Loaded Configuration File	/etc/php/8.3/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.3/apache2/conf.d
Additional .ini files parsed	/etc/php/8.3/apache2/conf.d/10-mysqld.ini, /etc/php/8.3/apache2/conf.d/10-opcache.ini, /etc/php/8.3/apache2/conf.d/10-pdo.ini, /etc/php/8.3/apache2/conf.d/15-xml.ini, /etc/php/8.3/apache2/conf.d/20-calendar.ini, /etc/php/8.3/apache2/conf.d/20-ctype.ini, /etc/php/8.3/apache2/conf.d/20-curl.ini, /etc/php/8.3/apache2/conf.d/20-dom.ini, /etc/php/8.3/apache2/conf.d/20-exif.ini, /etc/php/8.3/apache2/conf.d/20-ffi.ini, /etc/php/8.3/apache2/conf.d/20-fileinfo.ini, /etc/php/8.3/apache2/conf.d/20-ftp.ini, /etc/php/8.3/apache2/conf.d/20-gd.ini, /etc/php/8.3/apache2/conf.d/20-gettext.ini, /etc/php/8.3/apache2/conf.d/20-iconv.ini, /etc/php/8.3/apache2/conf.d/20-intl.ini, /etc/php/8.3/apache2/conf.d/20-mbstring.ini, /etc/php/8.3/apache2/conf.d/20-mysqli.ini, /etc/php/8.3/apache2/conf.d/20-pdo_mysql.ini, /etc/php/8.3/apache2/conf.d/20-phar.ini, /etc/php/8.3/apache2/conf.d/20-posix.ini, /etc/php/8.3/apache2/conf.d/20-readline.ini, /etc/php/8.3/apache2/conf.d/20-shmop.ini, /etc/php/8.3/apache2/conf.d/20-simplexml.ini, /etc/php/8.3/apache2/conf.d/20-sockets.ini, /etc/php/8.3/apache2/conf.d/20-sysmsg.ini, /etc/php/8.3/apache2/conf.d/20-sysvsem.ini, /etc/php/8.3/apache2/conf.d/20-sysvshm.ini, /etc/php/8.3/apache2/conf.d/20-tokenizer.ini, /etc/php/8.3/apache2/conf.d/20-xmireader.ini, /etc/php/8.3/apache2/conf.d/20-xmlwriter.ini, /etc/php/8.3/apache2/conf.d/20-xsl.ini, /etc/php/8.3/apache2/conf.d/20-zip.ini
PHP API	20230831
PHP Extension	20230831
Zend Extension	420230831

Hemos solucionado el error pero si probáramos abrir simplemente `localhost:80` nos sigue saliendo el error del antiguo trabajo



EL PROBLEMA ERA POR EL CACHE DEL NAVEGADOR, ABRIENDO EN MODO DE INCOGNITO O USANDO EL COMANDO DE ABAJO FUNCIONA CORRECTAMENTE

<http://wsl.localhost>



6.2 Configuración Wordpress

Entra a MySQL con el usuario root:

```
sudo mysql -u root -p
```

Crea el usuario kike con la contraseña kike123:

```
CREATE USER 'kike'@'localhost' IDENTIFIED BY 'kike123';
```

Da permisos sobre la base de datos wordpress:

```
GRANT ALL PRIVILEGES ON wordpress.* TO 'kike'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

Reinicia Apache:

```
sudo systemctl restart apache2
```

Hola

¡Este es el famoso proceso de instalación de WordPress en cinco minutos! Simplemente completa la información siguiente y estarás a punto de usar la más enriquecedora y potente plataforma de publicación personal del mundo.

Información necesaria

Por favor, proporciona la siguiente información. No te preocupes, siempre podrás cambiar estos ajustes más tarde.

Título del sitio

Nombre de usuario

Los nombres de usuario pueden tener únicamente caracteres alfanuméricos, espacios, guiones bajos, guiones medios, puntos y el símbolo @.

Contraseña

Ocultar

Fuerte

Importante: Necesitas esta contraseña para acceder. Por favor, guárdala en un lugar seguro.

Buscar

19/11

Título del sitio

ASIR

Nombre de usuario

admin o kike

Es el nombre visible de la página web. Puedes poner cualquier título.

Será tu usuario para iniciar sesión en el panel de administración (/wp-admin).

Contraseña

Puedes dejar la que te da WordPress (RtJF7N^QuH2E3r8QuS) o escribir la tuya.

Si usas la tuya (por ejemplo kike123), asegúrate de que sea segura y fácil de recordar.

Tu correo electrónico

tucorreo@ejemplo.com

WordPress lo usa para recuperar la contraseña o recibir notificaciones.

Visibilidad en los motores de búsqueda

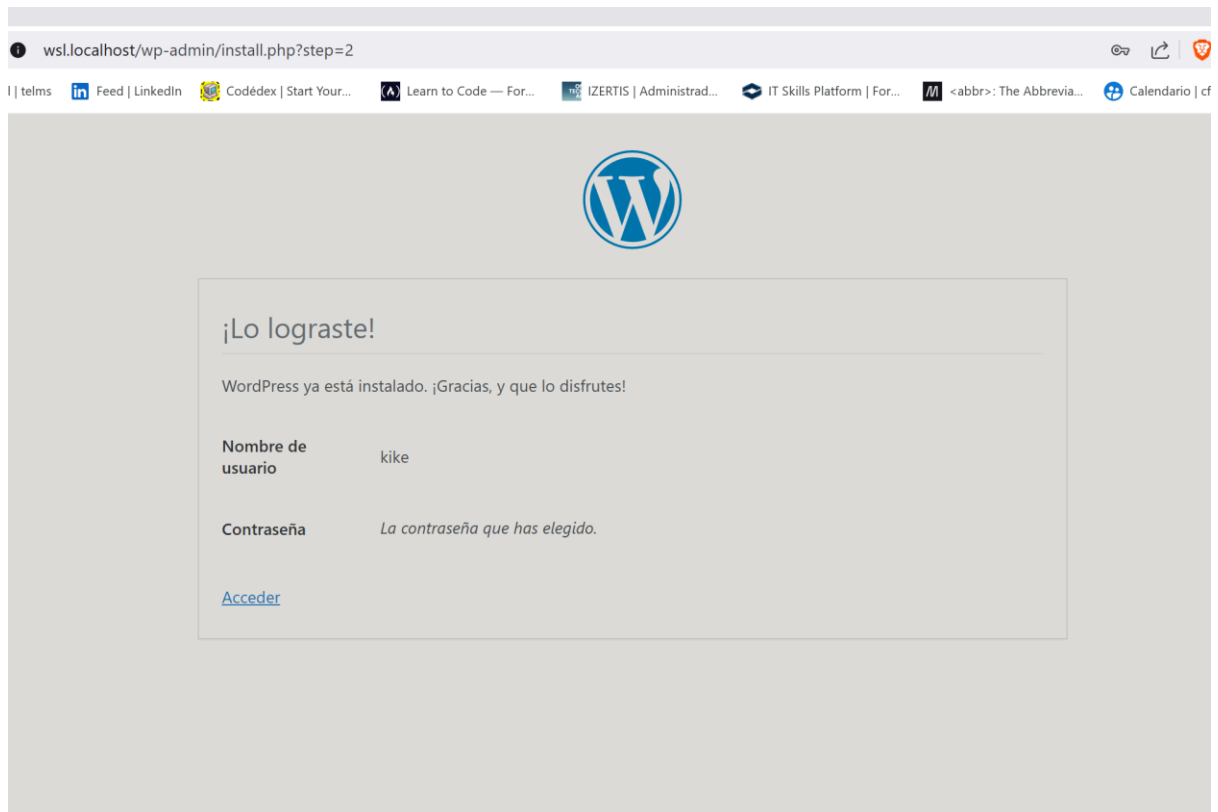
☐ No marcar (a menos que quieras ocultar el sitio).

Si la marcas, Google no indexará tu web.



+ Pregunta lo que quieras





7 – ¿Qué es Ngrok?

Ngrok es un servicio que crea un “túnel seguro” entre Internet y tu máquina local.

👉 Esto permite que otras personas (o tú desde otro dispositivo) accedan a tu WordPress local sin necesidad de:

- Abrir puertos en tu router.
- Configurar DNS o IP pública.
- Cambiar el cortafuegos.

♦ En resumen: Ngrok asigna una URL temporal tipo

<https://1234abcd.ngrok.io>

que redirige automáticamente al <http://localhost> de tu WSL.

7.1 – Instalación de Ngrok

1. Descarga Ngrok desde su web oficial:

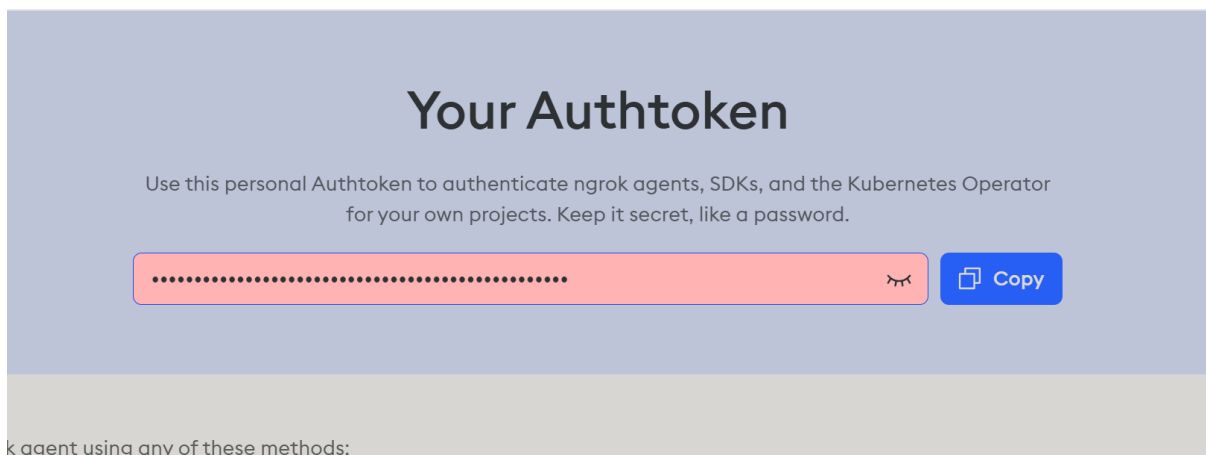
👉 <https://ngrok.com/download>

2. En Ubuntu (WSL), ejecuta:

```
kike@Double-KK:/mnt/c/Windows/System32$ sudo systemctl restart apache2
kike@Double-KK:/mnt/c/Windows/System32$ sudo snap install ngrok
2025-10-19T12:54:40+02:00 INFO Waiting for automatic snapd restart...
Automatically connect eligible plugs and slots of snap "snapd"
```

```
kike@Double-KK:/mnt/c/Windows/System32$ ngrok version
ngrok version 3.29.0
```

7.2 – Registro y obtención del token



7.3 – Configurar el token en tu entorno

Paso 3 – Pegar tu token en WSL

En tu terminal Ubuntu o WSL, ejecuta:

ngrok config add-authtoken TU_TOKEN_AQUI


```
Authtoken saved to configuration file: /home/kike/snap/ngrok/315/.config/ngrok/ngrok.yml
kike@Double-KK:/mnt/c/windows/System32$
```

7.4 – Ejecutar el túnel hacia Apache

Iniciar ngrok en el puerto 80

Tu WordPress está sirviendo por Apache en el puerto 80 (puedes verificarlo con `sudo apache2ctl -S`), así que ejecuta:

```
kike@Double-KK: /mnt/c/Windows/System32
ngrok

🔗 Create instant endpoints for local containers within Docker Desktop → https://ngrok.com/r/docker

Session Status      online
Account             kike (Plan: Free)
Update              update available (version 3.30.0, Ctrl-U to update)
Version             3.29.0
Region              Europe (eu)
Latency             29ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://markus-nonpacifiable-dereistically.ngrok-free.dev -> http://localhost:80

Connections          ttl    opn    rt1    rt5    p50    p90
                    1      0      0.01   0.00   5.16   5.16

HTTP Requests
-----
13:05:01.630 CEST GET /                200 OK
13:05:01.785 CEST GET /favicon.ico     404 Not Found
```

ngrok generó una URL pública:

<https://markus-nonpacifiable-dereistically.ngrok-free.dev>

Al acceder a esa URL, se pudo visualizar y administrar el sitio de WordPress de forma remota, validando que el servicio estaba funcionando correctamente.



8 – Conclusiones y Reflexión Final

◇ Resumen del proyecto

En este proyecto se ha conseguido **instalar y configurar WordPress en WSL2 (Ubuntu)**, utilizando el stack completo **LAMP (Linux, Apache, MySQL y PHP)**. Posteriormente, se logró **hacer el sitio accesible desde Internet** mediante **ngrok**, un túnel seguro que evita la necesidad de abrir puertos en el router.

◇ Fases principales realizadas

1. Configuración del entorno LAMP

- Instalación de Apache, PHP 8.3 y MySQL en Ubuntu sobre WSL2.
- Verificación de módulos y servicio Apache activos mediante comandos como `sudo systemctl status apache2` y `php -v`.
- Creación del archivo `phpinfo.php` para confirmar la ejecución de PHP.

2. Despliegue de WordPress

- Descarga y extracción del paquete de WordPress en `/var/www/html`.
- Creación de la base de datos wordpress y del usuario kike con permisos adecuados.
- Configuración del archivo `wp-config.php` con las credenciales:

```
DB_NAME = wordpress
DB_USER = kike
DB_PASSWORD = kike123
DB_HOST = localhost
```

- d. Inicio de la instalación desde el navegador y creación del usuario administrador del sitio.

3. Publicación en Internet con ngrok

- a. Creación de una cuenta gratuita en ngrok.com.
- b. Obtención del token de autenticación personal (**Authtoken**) desde el panel de usuario.
- c. Ejecución del comando:

```
ngrok config add-authtoken <mi_token>
ngrok http 80
```

- d. ngrok generó una URL pública segura (HTTPS) como:

<https://markus-nonpacifiable-dereistically.ngrok-free.dev>

- e. Al abrirla desde cualquier dispositivo externo, el sitio WordPress fue accesible públicamente.

◇ Resultados obtenidos

- WordPress instalado y funcionando correctamente en entorno local (WSL2).
- Sitio accesible desde Internet sin abrir puertos.
- Configuración estable de Apache, MySQL y PHP 8.3.
- Validación de conectividad, permisos y módulos activos.
- Éxito en la publicación mediante túnel ngrok.

◇ Dificultades enfrentadas y soluciones

Problema	Causa	Solución aplicada
----------	-------	-------------------

El navegador mostraba “Hola Apache desde 8080...” en lugar de WordPress.	Apache servía otro archivo index o puerto.	Se eliminó <code>index.html</code> y se ajustó el <code>DocumentRoot</code> en <code>/etc/apache2/sites-enabled/000-default.conf</code> .
Error de conexión a la base de datos.	WordPress no podía autenticar con MySQL.	Se corrigió el usuario y contraseña en <code>wp-config.php</code> y se verificó la base con <code>mysql -u root -p</code> .
“phpinfo.php not found”	Archivo mal ubicado o permisos.	Se creó correctamente en <code>/var/www/html</code> y se ajustaron permisos con <code>chown www-data:www-data</code> .
ngrok no conectaba al iniciar	Faltaba token o configuración.	Se ejecutó <code>ngrok config add-authtoken</code> y se inició el túnel manualmente.

◆ Reflexión personal

Esta práctica permitió comprender cómo **funciona el entorno LAMP** dentro de WSL2 y cómo **WordPress gestiona su base de datos y archivos internos**.

Además, se aprendió a **publicar un sitio web de desarrollo en Internet** usando ngrok, una herramienta moderna que facilita pruebas y demostraciones sin depender de configuraciones de red complejas.

Ha sido especialmente útil para afianzar conceptos de:

- Administración de servicios Linux.
- Permisos de archivos en Apache.
- Configuración de PHP y MySQL.
- Seguridad y exposición controlada mediante túneles HTTPS.

◆ Conclusión

Se ha completado con éxito el despliegue de WordPress sobre WSL2, validando su integración con Apache, MySQL y PHP, y demostrando su accesibilidad pública mediante ngrok.

El proyecto cumple con todos los objetivos planteados, proporcionando un entorno funcional, seguro y replicable para futuros entornos de desarrollo web profesional.

